



IŞIK UNIVERSITY FACULTY OF
ADMINISTRATIVE ECONOMICS
AND SOCIAL SCIENCES
Department of Management
Information Systems

AI-BASED FINANCIAL ANOMALY DETECTION AND RISK SCORING SYSTEM WITH MICROSERVICES ARCHITECTURE

A Dual-Task Ensemble Learning Approach with Event-Driven
Communication

Emir Karadeniz

A Bachelor's Thesis

Faculty of Economics, Administrative and Social Sciences
Department of Management Information Systems

Supervisor: Dr. Şahin Aydın

JUNE 2026

AI-Based Financial Anomaly Detection and Risk Scoring System

with Microservices Architecture

A Project Presented

by

Emir Karadeniz

22MISY1034

to

The Department of Management Information Systems

in partial fulfillment of the requirements

for the degree of

Bachelor of Science

in the field of

Management Information Systems

Işık University

İstanbul, Turkey

June 2026

© Copyright 2026 by Emir Karadeniz

All Rights Reserved

This thesis is the outcome of a long journey of learning, perseverance, and growth. It is dedicated to my family, who have always believed in me and supported me unconditionally, and to my friends, who kept me motivated and inspired throughout this journey.

Abstract

Financial transaction monitoring faces two parallel challenges: single-algorithm machine learning detectors and static rule-based risk scoring cannot adapt to the diversity of anomalous behavior or the evolution of user profiles, while monolithic implementations couple detection logic to transaction processing in ways that constrain independent retraining, scaling, and replacement.

This thesis presents AFRATS, an AI-based financial anomaly detection and risk scoring system that addresses both challenges within a single integrated platform. The backend decomposes into five independently deployable services — AuthService, TransactionService, MLService, NotificationService, and an API Gateway — built on ASP.NET Core 10 following Clean Architecture and communicating asynchronously through a RabbitMQ event bus. The machine learning layer is a dedicated Python/FastAPI service exposing two complementary tasks: a transaction-level anomaly detection ensemble combining Z-score, Isolation Forest, Local Outlier Factor, and XGBoost, and a user-level risk classification ensemble combining Random Forest, XGBoost, and Logistic Regression. Both inference paths are paired with an explicit business rule override layer and a persistent audit trail.

The system is evaluated on a synthetic dataset of 694,327 transactions across 2,000 users, calibrated to Turkish household consumption patterns. On the test set, the anomaly detection ensemble achieves F1 0.813 (precision 0.733, recall 0.914), and the risk classifier reaches 81.25% accuracy with 0.761 macro F1 under clean-label evaluation. End-to-end validation confirms the full event flow from transaction creation to notification delivery.

Keywords: microservices architecture, financial anomaly detection, risk scoring, event-driven systems, ensemble learning

Acknowledgements

I would like to express my sincere gratitude to my thesis advisor, Dr. Şahin Aydın, for his invaluable guidance, constructive feedback, and continuous support throughout the development of this thesis. His vision and mentorship shaped both the academic direction and the technical perspective of this work.

I would also like to thank Deniz Altıntaş for her support during the technical and methodological stages of the thesis, as well as for the knowledge and contributions she generously shared throughout the process.

My sincere appreciation also goes to Işık University and the Management Information Systems department for the academic foundation and broad perspective they provided over the past four years. I am deeply grateful to all my professors, whose insights and dedication contributed significantly to my development both technically and academically.

Finally, I would like to thank my family and friends for their unwavering support, patience, and encouragement throughout this demanding period.

Contents

Abstract.....	iv
Acknowledgements.....	v
Contents.....	vi
List of Figures.....	viii
List of Tables.....	ix
Abbreviations.....	x
 CHAPTER 1 AIM AND SCOPE.....	 1
1.1 Introduction.....	2
1.2 Motivation.....	2
1.3 Contributions.....	2
1.4 Outline of the Project.....	3
 CHAPTER 2 LITERATURE REVIEW.....	 4
2.1 Related Work.....	5
2.2 Research Gap.....	7
 CHAPTER 3 METHODOLOGY.....	 9
3.1 Synthetic Dataset Construction.....	10
3.2 Anomaly Detection Algorithm Design.....	14
3.3 Risk Classification Algorithm Design.....	17
3.4 System Architecture.....	22
3.5 Data Model.....	23
3.6 Event-Driven Communication Design.....	25
 CHAPTER 4 IMPLEMENTATION.....	 28
4.1 Software Stack and Cross-Cutting Patterns.....	29
4.2 AuthService.....	30
4.3 TransactionService.....	30
4.4 MLService.....	31
4.5 Notification.....	31
4.6 Gateway Layer.....	32

4.7	Container Orchestration.....	32
4.8	Frontend Application.....	33
CHAPTER 5 EVALUATION AND RESULTS.....		35
5.1	Results.....	36
5.2	Discussion.....	43
CHAPTER 6 CONCLUSION AND FUTURE DIRECTIONS.....		45
6.1	Conclusion.....	46
6.2	Limitations.....	46
6.3	Future Work.....	47
	References.....	48

List of Figures

Figure 1. Three-stage synthetic dataset generation pipeline.....	10
Figure 2. Anomaly type distribution in the generated dataset.....	13
Figure 3. Anomaly detection ensemble decision flow.....	16
Figure 4. Risk classification decision flow.....	21
Figure 5. AFRATS service topology.....	23
Figure 6. RiskScores.Factors JSON payload (rule-influenced score).....	24
Figure 7. Cross-database entity relationships.....	25
Figure 8. End-to-end event flow sequence.....	27
Figure 9. Dashboard view on the web client.....	34
Figure 10. Dashboard view on the React Native mobile client.....	34
Figure 11. Per-algorithm F1 and MCC comparison.....	37
Figure 12. Confusion matrices for the four weight profiles.....	38
Figure 13. Confusion matrix for the deployed ensemble (clean-label evaluation).....	40
Figure 14. Random Forest Gini feature importance.....	41
Figure 15. RiskScores rows for a single user.....	42

List of Tables

Table 1. Income band partitioning anchored to the 2026 Turkish minimum wage.....	11
Table 2. Expense category weights used in synthetic data generation.....	11
Table 3. Behavioral parameter distributions by risk class.....	12
Table 4. Anomaly injection parameters by type.....	13
Table 5. Data split structure for the anomaly detection and risk classification tasks.....	14
Table 6. Anomaly detection feature representation.....	15
Table 7. Risk classification feature representation.....	18
Table 8. Business rule override layer.....	20
Table 9. RabbitMQ broker topology.....	26
Table 10. Implementation libraries by Clean Architecture layer.....	29
Table 11. AFRATS container inventory and orchestration metadata.....	33
Table 12. Per-algorithm metrics on the test set.....	36
Table 13. Ensemble strategies and weight profile metrics.....	37
Table 14. Per-classifier metrics on the noisy-label test set.....	39
Table 15. Ensemble weight profile metrics on the noisy-label test set.....	39
Table 16. End-to-end latency statistics.....	42

Abbreviations

AFRATS: AI-based Financial Anomaly Detection and Risk Scoring system

AI: Artificial Intelligence

AMQP: Advanced Message Queuing Protocol

API: Application Programming Interface

BCrypt: Blowfish-based Cryptographic Hash Function

CORS: Cross-Origin Resource Sharing

CPU: Central Processing Unit

CQRS: Command Query Responsibility Segregation

DTO: Data Transfer Object

EF: Entity Framework

F1: F1 Score (harmonic mean of precision and recall)

HMAC: Hash-based Message Authentication Code

HTTP: HyperText Transfer Protocol

IF: Isolation Forest

JSON: JavaScript Object Notation

JWT: JSON Web Token

LOF: Local Outlier Factor

LR: Logistic Regression

MCC: Matthews Correlation Coefficient

ML: Machine Learning

OAuth: Open Authorization

PR-AUC: Precision-Recall Area Under Curve

RF: Random Forest

RFC: Request for Comments

ROC-AUC: Receiver Operating Characteristic Area Under Curve

SHA: Secure Hash Algorithm

SMOTE: Synthetic Minority Over-sampling Technique

SMTP: Simple Mail Transfer Protocol

SQL: Structured Query Language

TRY: Turkish Lira

TÜİK: Türkiye İstatistik Kurumu (Turkish Statistical Institute)

XGBoost: eXtreme Gradient Boosting

YARP: Yet Another Reverse Proxy

Chapter 1

Aim and Scope

This chapter introduces the research problem, summarizes the main contributions, and frames the structure of the thesis.

1.1 Introduction

The digitalization of financial services has reshaped both the scale at which transactions are processed and the architectural complexity of the surrounding systems. Modern payment ecosystems handle millions of transactions per day across distributed networks, requiring large-scale behavioral analysis alongside backends that can evolve under continuous development pressure. Two parallel concerns emerge: detecting anomalous behavior at the transaction and user level, and organizing the architecture so detection logic can be retrained, scaled, or replaced independently of authentication and transaction processing. This thesis develops the Artificial Intelligence (AI)-based Financial Anomaly Detection and Risk Scoring System (AFRATS), which treats architectural decoupling, transaction-level anomaly detection, user-level risk scoring, and transparent dual-regime evaluation on a purpose-built synthetic dataset as four equally important design concerns within a single, end-to-end validated platform.

1.2 Motivation

Current approaches address the architectural, detection, scoring, and evaluation dimensions of the problem in isolation. Monolithic implementations bind inference to transaction processing, preventing independent retraining. Single-algorithm detectors cover only a subset of anomaly structures, while static rule-based scoring cannot adapt to behavioral evolution. Evaluation rests on datasets that rarely guard against feature-label leakage, cross-user contamination, or unbounded annotation noise. The central proposition is that these limitations are most usefully addressed together; the gap analysis is developed in Chapter 2.

1.3 Contributions

This thesis makes the following main contributions.

- Five-service event-driven microservices architecture. A backend of AuthService, TransactionService, MLService, NotificationService, and an Application Programming Interface (API) Gateway decomposes the system into independently deployable bounded contexts communicating asynchronously through a RabbitMQ event bus. Each ASP.NET Core service follows Clean Architecture principles, while the Machine Learning (ML) layer runs as

a dedicated Python/FastAPI service that can be retrained without affecting the .NET transactional core.

- Four-algorithm hybrid anomaly detection ensemble. A weighted ensemble combines Z-score, Isolation Forest (IF), Local Outlier Factor (LOF), and XGBoost (eXtreme Gradient Boosting) with complementary inductive biases. The XGB-heavy profile (0.30 / 0.20 / 0.45 / 0.05) achieves test-set F1 score (F1) = 0.813 with precision 0.733, recall 0.914, and Matthews Correlation Coefficient (MCC) = 0.809.

- Three-algorithm supervised risk classification ensemble. A separate user-level ensemble combines Random Forest (RF), XGBoost, and Logistic Regression (LR) (weights 0.20 / 0.70 / 0.10), producing three discrete risk levels and a normalized 0–100 continuous score. Under clean-label evaluation it reaches 81.25% accuracy with macro F1 0.761.

- Model-versus-rule disentanglement with a purpose-built synthetic dataset. The system treats learned ensembles and business rule overrides as separate auditable sources, persisting an override-reasons audit trail. A Turkish Statistical Institute (TÜİK)-calibrated synthetic dataset of 694,327 transactions across 2,000 users supports user-level stratified splitting and controlled label noise.

1.4 Outline of the Project

Chapter 2 reviews the literature across four domains and concludes with a research gap analysis. Chapter 3 presents the methodology and system design, covering synthetic dataset construction, ensemble design, system architecture, the data model, and event-driven communication. Chapter 4 describes the implementation. Chapter 5 reports the experimental results and discussion, including end-to-end validation. Chapter 6 concludes with findings, limitations, and future work.

Chapter 2

Literature Review

This chapter reviews the literature relevant to AFRATS across four domains — financial anomaly detection, machine learning for credit risk classification, microservices architecture, and event-driven communication — and concludes with a research gap analysis.

2.1 Related Work

The four domains group into two sides. The machine learning side covers anomaly detection in financial transactions and supervised credit risk classification — motivating the dual-task inference paths exposed by a single ML microservice. The architectural side covers microservices design in financial services and event-driven communication — motivating the five-service decomposition, the Yet Another Reverse Proxy (YARP) gateway, and the RabbitMQ event bus.

On the machine learning side, rule-based fraud detection has come under increasing pressure as transaction volumes grow, motivating the shift toward AI-driven approaches. Sarna et al. (2025) provide a systematic review of 24 papers and conclude that supervised methods (Random Forest, gradient boosting) and unsupervised methods (Isolation Forest, LOF) address distinct facets of the detection problem, and that ensemble combinations consistently outperform single-model implementations. Alabi and Ang (2024) reinforce this with a focused study on Z-score, Isolation Forest, one-class Support Vector Machine, and deep outlier detection, reporting that ensembles improve both accuracy and operational reliability — the methodological case AFRATS adopts through its four-algorithm pipeline.

Within the unsupervised family, John and Naaz (2019) compare Isolation Forest and LOF on the 284,807-record European credit card dataset, finding LOF at 97% and Isolation Forest at 76% accuracy alongside supervised baselines, while emphasizing that precision-recall evaluation is essential on imbalanced data. Fadul (2023) extends this across seven datasets, finding Isolation Forest tends to outperform LOF in AUC and speed on large data while LOF achieves stronger MCC where local density structures dominate — supporting their combined inclusion. The supervised component is grounded in Purwar and Manju (2023), who evaluate XGBoost on the European credit card dataset, reporting $F1 = 82.78\%$ with class-weight adjustment rather than oversampling — motivating the AFRATS use of `scale_pos_weight` rather than Synthetic Minority Over-sampling Technique (SMOTE). Shanaa and Abdallah (2025) study supervised-unsupervised combinations directly, reporting $F1 = 0.941$ by combining an unsupervised autoencoder with a supervised XGBoost classifier through an optimized thresholding scheme — a finding that parallels the AFRATS weighting scheme.

Beyond transaction-level anomaly detection, financial risk assessment requires a supervised component at the user level: where anomaly detection asks whether a transaction deviates from expected behavior, risk classification asks how a user’s cumulative profile maps

to a tier and a continuous score. Sinaga and Machfud (2025) provide direct empirical support for the AFRATS supervised component, comparing Random Forest and XGBoost on the German Credit Dataset and reporting XGBoost outperforming on accuracy (0.91 vs 0.88), F1 (0.89 vs 0.85), and area under the receiver operating characteristic curve (ROC-AUC) (0.94 vs 0.90) — a result that directly shapes the XGBoost-dominant weighting ($w_{\text{XGB}} = 0.70$) in the AFRATS ensemble. Odunlami and Nwonu (2025) compare six algorithms within a loan-default pipeline pairing SMOTEENN with Principal Component Analysis, finding gradient-boosted ensembles consistently outperform linear classifiers on nonlinearly structured financial features. Li et al. (2025) integrate Long Short-Term Memory and transformer architectures with gradient boosting, reaching AUC-ROC of 0.971 and reducing false positive rates by 34.7% over a rule-based baseline — corroborating the value of gradient-boosted classifiers within hybrid architectures for financial risk signals. The dual-regime (noisy- versus clean-label) evaluation and the deployment-time tunable 0–100 score are AFRATS’s own methodological choices rather than recommendations attributable to this source.

On the architecture side, the decision to implement AFRATS as distributed microservices rather than a monolith is grounded in empirical work on financial service decomposition. Dhandapani (2025) reports that financial institutions adopting microservices achieve deployment cycles 8–10 times more frequent than monolithic equivalents. Marieska et al. (2025) provide controlled performance data showing that microservices deliver roughly 36% faster response times and 71% fewer errors up to 450 concurrent users, with overhead beginning to limit throughput beyond 90% Central Processing Unit (CPU) utilization — contextualizing the deployment envelope within which AFRATS’s five-service decomposition operates.

Internally, AFRATS combines Clean Architecture with Command Query Responsibility Segregation (CQRS) through MediatR. Yakhin (2025) compares Clean Architecture and Vertical Slice Architecture in enterprise .NET environments, arguing that hybrid configurations balance domain stability with deployment flexibility. Pinciroli et al. (2023) provide queuing-network models showing CQRS — by isolating reading from updating — simplifies database design and allows command and query workloads to scale independently, directly relevant where transactional writes and aggregated ML reads have distinct profiles. For cross-service identity, de Almeida and Canedo (2022) survey Open Authorization (OAuth) 2.0, OpenID Connect, JavaScript Object Notation (JSON) Web Token (JWT), and API Gateway-mediated propagation, reaching the consensus that an API gateway

combined with stateless JWT tokens validated centrally is the mechanism most consistently identified as appropriate — mapping directly onto the AFRATS YARP-based Gateway.

The asynchronous communication layer between AFRATS microservices is grounded in research on event-driven patterns in fintech. Odofoin et al. (2022) compare Apache Kafka and RabbitMQ: Kafka’s distributed log-based streaming is best suited to high-throughput and message-replay scenarios, while RabbitMQ’s queue-based Advanced Message Queuing Protocol (AMQP) model fits low-latency transactional delivery and command-driven workflows — which underpins the AFRATS messaging choice given moderate event volume and command-driven flow. Laigner et al. (2025) mine 628 Stack Overflow questions on event management in microservices and identify sixteen challenge categories spanning safety, schema management, performance, observability, and security; weak delivery semantics account for 17.20% alone. AFRATS responds by treating idempotency at the inference-result level: before persisting, MLService performs a transaction-scoped existence check and skips the write if anomaly logs already exist, with concurrency safety provided by a single-consumer RabbitMQ topology (`prefetch_count = 1`).

2.2 Research Gap

The literature documents substantial progress across four domains — ensemble ML methods for anomaly detection (Sarna et al., 2025; John & Naaz, 2019; Fadul, 2023; Purwar & Manju, 2023; Shanaa & Abdallah, 2025) and risk classification (Sinaga & Machfud, 2025; Li et al., 2025; Odunlami & Nwonu, 2025); microservices decomposition (Dhandapani, 2025; Marieska et al., 2025; Yakhin, 2025; Pincioli et al., 2023; de Almeida & Canedo, 2022); and event-driven communication (Odofoin et al., 2022; Laigner et al., 2025). Four gaps remain that AFRATS addresses: (i) architectural studies of financial microservices validate decomposition under simplified detection logic, not dual-task ensemble inference; (ii) transaction-level anomaly detection and user-level risk classification are typically studied separately, with no shared microservice exposing both; (iii) the disentanglement of learned model decisions from business rule overrides receives little empirical attention; and (iv) existing literature relies on a small set of widely reused public datasets with little explicit attention to feature-label leakage, user-level stratified splitting, and controlled label noise.

AFRATS responds through an integrated design: a five-service Clean Architecture .NET backend with RabbitMQ messaging, dual-task ensemble inference within a

single ML microservice, business rule override layers with persistent audit trails, and a TÜİK-calibrated synthetic dataset of 694,327 transactions across 2,000 users under user-level stratified split with controlled label noise.

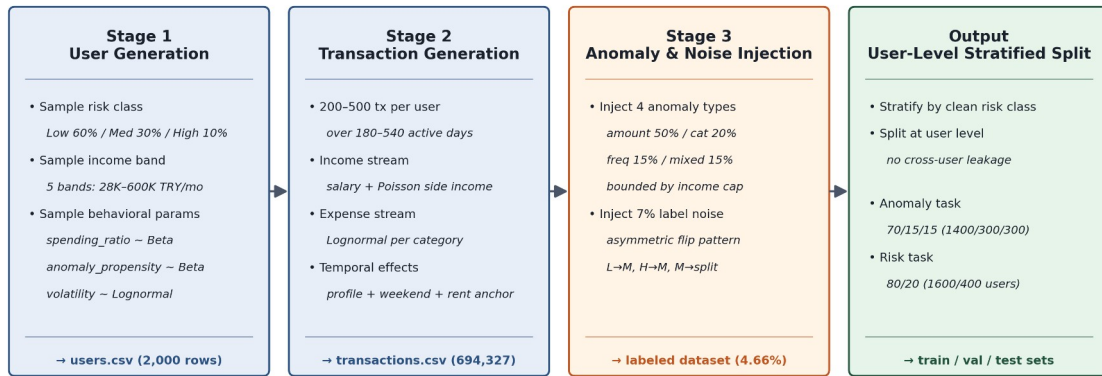
Chapter 3

Methodology

This chapter presents the methodology of AFRATS, covering the synthetic dataset, the algorithm design for both inference paths, the system architecture, the data model, and the event-driven communication design.

3.1 Synthetic Dataset Construction

The evaluation of user-personalized anomaly detection and risk scoring requires a dataset constructed for that purpose. Public benchmarks such as the Credit Card Fraud Detection Dataset contain anonymized real transactions with limited behavioral context, while third-party synthetic generators produce distributions misaligned with a specific deployment context. An earlier AFRATS version relied on the Sparkov generator but was deprecated because its structural assumptions conflict with the user-personalized spending anomaly target of this thesis. A dedicated generator was developed in `scripts/generate_synthetic_dataset.py`, fully deterministic when seeded with `SEED = 42`, producing 2,000 simulated users and 694,327 transactions. The outputs `users.csv` and `transactions.csv` are uniquely identified by Secure Hash Algorithm (SHA)-256 digests, supporting reproducibility.



All randomness seeded by `SEED = 42` — deterministic reproducibility across runs

Figure 1. Three-stage synthetic dataset generation pipeline.

Calibration and user generation. The base income distribution is anchored to the Turkish minimum wage as of 2026 and partitioned into five bands, as shown in Table 1; the two highest bands are lightly oversampled to ensure statistical reliability of evaluation metrics on the higher-income segment. The ten expense categories used here — Rent, Bills, Grocery, Transportation, Health, Clothing, Entertainment, Education, Electronics, and Other Expense (Table 2) — are a project-defined scheme that does not exactly match the official TÜİK classification. Their proportions were loosely calibrated to the aggregate household consumption shares published by the Turkish Statistical Institute for 2024 (Türkiye İstatistik Kurumu, 2025), so that overall spending is approximately realistic; the dataset itself is fully synthetic and contains no TÜİK data. All amounts are in Turkish Lira (TRY).

Band	Monthly Income Range (TRY)
1	28,000 – 45,000
2	45,000 – 80,000
3	80,000 – 150,000
4	150,000 – 300,000
5	300,000 – 600,000

Table 1. Income band partitioning anchored to the 2026 Turkish minimum wage.

Expense Category	Weight
Rent	0.180
Bills	0.100
Grocery	0.200
Transportation	0.130
Health	0.060
Clothing	0.070
Entertainment	0.100
Education	0.040
Electronics	0.050
Other Expense	0.070
Total	1.000

Table 2. Expense category weights used in synthetic data generation.

Each user is assigned a risk class sampled from a categorical distribution (60% Low, 30% Medium, 10% High) and constructed from behavioral parameters drawn from class-dependent distributions, summarized in Table 3. The overlap between class ranges structurally prevents feature-label leakage, since a Low-risk user with high volatility and a Medium-risk user with low volatility can produce similar feature vectors.

Parameter	Distribution	Low Risk	Medium Risk	High Risk
Spending ratio	$\text{Beta}(\alpha, \beta)$	(2, 6), mean 0.250	(3, 3), mean 0.500	(6, 2), mean 0.750
Debt ratio	$\text{Beta}(\alpha, \beta)$	(2, 8), mean 0.200	(3, 4), mean 0.429	(5, 3), mean 0.625
Anomaly propensity	$\text{Beta}(\alpha, \beta)$	(1, 45), mean 0.022	(2, 28), mean 0.067	(3, 17), mean 0.150
Spend volatility	$\text{Normal}(\mu, \sigma)$, clipped [0.05, 0.80]	(0.13, 0.05)	(0.30, 0.09)	(0.50, 0.13)

Table 3. Behavioral parameter distributions by risk class.

Transaction generation and anomaly injection. Each user is assigned 200–500 transactions over 180–540 days. Income transactions are emitted separately from expenses: a monthly salary at the start of each month plus additional income types following per-user Poisson processes. Expense transactions are sampled from a per-category Lognormal distribution whose mean derives from income, spending ratio, and base category weight, and whose standard deviation is set by user volatility. Three temporal effects modulate the stream: per-user spending profile (Stable, Volatile, Accelerating), a weekend bias for Grocery/Entertainment/Clothing, and a fixed monthly anchor for Rent. After baseline generation, an injection stage marks a fraction of transactions as anomalous based on each user’s anomaly_propensity. Four anomaly types are injected with category-specific multipliers, all bounded by a per-user income cap ($1.499 \times$ monthly income). Table 4 reports the injection parameters; Figure 2 visualizes the resulting type distribution.

Anomaly Type	Probability	Magnitude Multiplier	Behavioral Constraint
Amount	50%	$[5.0, 10.0] \times \text{category mean}$	Capped at $1.499 \times \text{monthly income}$
Category	20%	$[2.5, 4.5] \times \text{user overall mean}$	Reassign to rarely-used category
Frequency	15%	$[2.0, 4.0] \times \text{category mean}$	≥ 60 days since last transaction in category
Mixed	15%	$[8.0, 15.0] \times \text{user overall mean}$	Rare category + income cap

Table 4. Anomaly injection parameters by type.

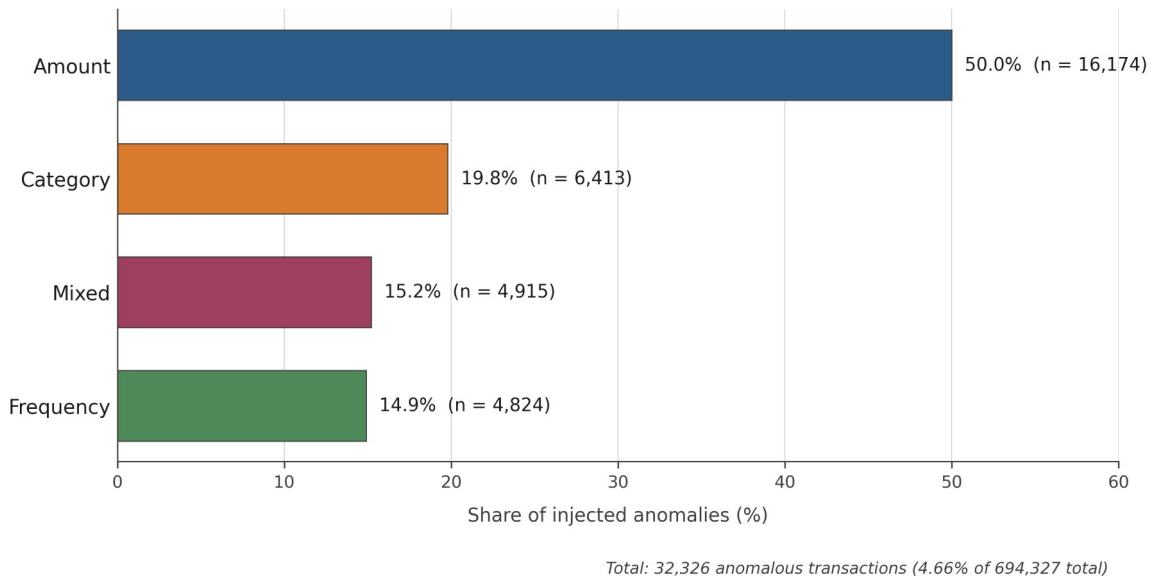


Figure 2. Anomaly type distribution in the generated dataset.

Label noise and split protocol. To simulate real-world annotation uncertainty, 7% of user risk-class labels are deliberately flipped during generation following an asymmetric pattern (Low $\rightarrow$ Medium, High $\rightarrow$ Medium, Medium splits at random) that approximates a plausible annotator hedging pattern (conservative regression toward the middle class). The original clean labels are retained alongside the noisy labels, enabling the dual-regime evaluation discussed in Section 5.1. User-level stratified splitting is applied with task-specific proportions (Table 5): both protocols stratify by the original clean risk class and split at the user level rather than the transaction level. All transactions of a given user are placed in the

same split, eliminating cross-user information leakage that would otherwise allow a model to memorize per-user behavioral patterns at training.

Task	Split Protocol	Train	Validation	Test	Total
Anomaly Detection (expense-filtered)	70 / 15 / 15	1,400 users 465,714 tx	300 users 96,306 tx	300 users 98,536 tx	2,000 users 660,556 tx
Risk Classification (user-level features)	80 / 20	1,600 users	—	400 users	2,000 users

Table 5. Data split structure for the anomaly detection and risk classification tasks.

3.2 Anomaly Detection Algorithm Design

The anomaly detection component is implemented as a weighted ensemble of four detectors over a shared 12-feature representation computed at request time from the user’s recent history. The ensemble combines a rule-based statistical detector (Z-score), two unsupervised learners (Isolation Forest and Local Outlier Factor), and a supervised gradient-boosting detector (XGBoost). The four are deliberately chosen for complementary inductive biases — each captures a family of behavior the others tend to miss — and their scores aggregate under a single weighted threshold rule. The 12-dimensional feature vector groups into three families (Table 6): six capture the transaction against the user’s recent behavior, one captures rolling temporal context, and five capture the relationship between the transaction’s category and the user’s behavior in that category. All features are computed on expense transactions only.

#	Feature	Description
1	amt	Raw transaction amount (TRY)
2	amount_zscore	Standardized deviation from user mean
3	amount_log_ratio	$\log(\text{amount} / \text{user_mean})$
4	user_mean	User's mean expense amount
5	user_std	User's std-dev of expense amount
6	user_median	User's median expense amount
7	daily_avg_7d	7-transaction rolling mean
8	category_zscore	Standardized deviation within category
9	category_freq	Fraction of user's expenses in this category
10	category_mean	User's mean expense in this category
11	cat_to_user_ratio	$\text{category_mean} / \text{user_mean}$
12	days_since_last_in_category	Days since user's last transaction in this category

Table 6. Anomaly detection feature representation.

Unsupervised and statistical detectors. The Z-score detector is a deterministic rule-based component requiring no training, computing the standardized deviation of amount from the user's historical mean with score $\min(|z| / 5.0, 1.0)$. Isolation Forest is trained over the full 12-feature representation with $n_estimators = 150$ and $max_features = 0.8$; the contamination parameter is set at runtime to the empirical anomaly rate of the training set (≈ 0.049). Local Outlier Factor is configured with $n_neighbors = 100$, $contamination = 0.07$, and $novelty = \text{True}$; unlike the runtime-empirical contamination of Isolation Forest, this fixed value was selected by grid search and held constant. a StandardScaler is fitted on the training set to prevent high-range features from dominating distance computation. Both unsupervised outputs are mapped to the unit interval through logistic compression.

Supervised XGBoost detector. XGBoost is the only supervised component, trained on a 7-feature subset of the full representation with $n_estimators = 400$, $max_depth = 6$, $learning_rate = 0.08$, $subsample = 0.9$, $colsample_bytree = 0.9$, $min_child_weight = 3$, $reg_lambda = 1.5$, and $scale_pos_weight \approx 19.3$ (the empirical negative-to-positive ratio). SMOTE-style oversampling is avoided on an already synthetic dataset, consistent with Purwar and Manju (2023).

Ensemble decision rule. Detector scores aggregate as:

$$ensemble_score = w_Z \cdot s_Z + w_{IF} \cdot s_{IF} + w_{XGB} \cdot s_{XGB} + w_{LOF} \cdot s_{LOF}$$

with deployed weights (0.30, 0.20, 0.45, 0.05); a transaction is flagged when $ensemble_score > 0.5$. The XGB-heavy weight reflects each detector's marginal contribution, consistent with the supervised-unsupervised hybrid of Shanaa and Abdallah (2025). Comparative results from grid search across alternative configurations are reported in Section 5.1. Feature extraction (app/ml/feature_engineering.py) and the anomaly ensemble logic (app/ml/anomaly_models.py) are reused without modification by both training and live inference; the trained components are serialized into models/anomaly_model.pkl loaded at startup.

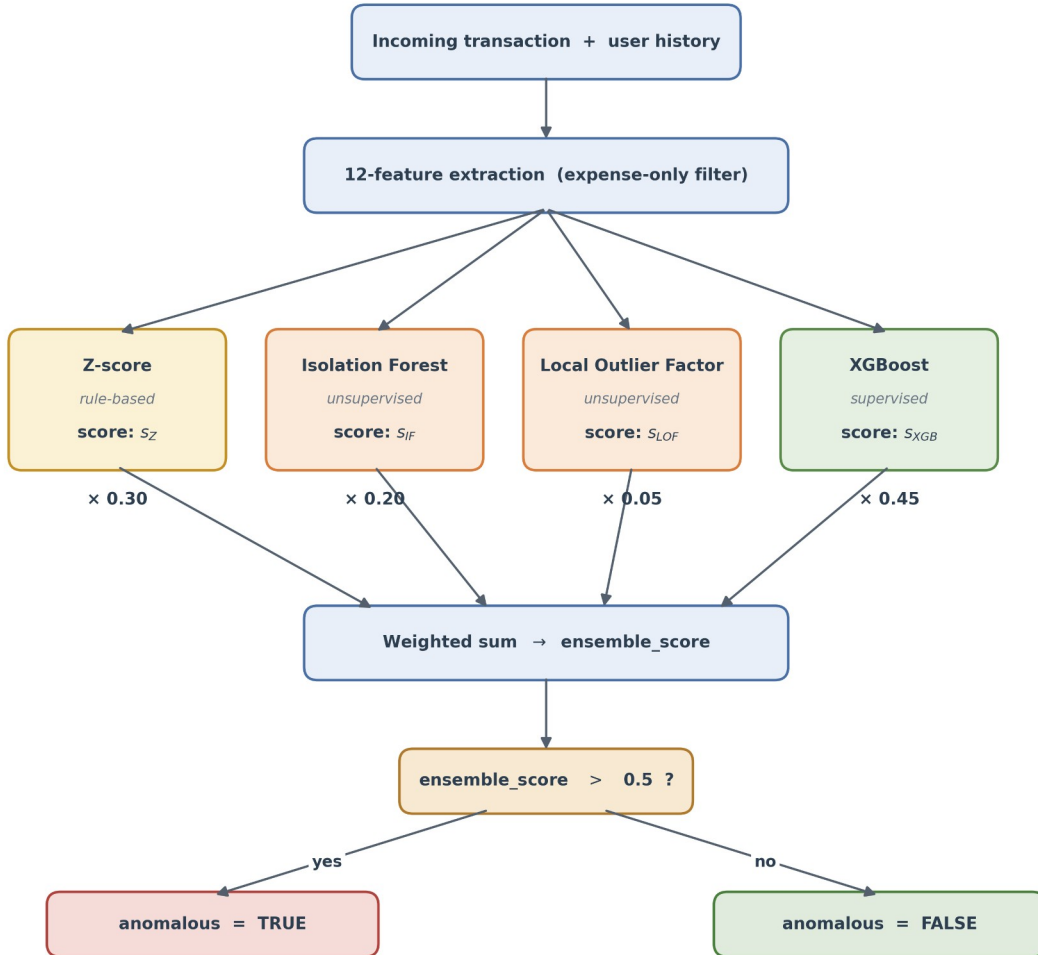


Figure 3. Anomaly detection ensemble decision flow.

3.3 Risk Classification Algorithm Design

The risk classification component operates at the user level: the classifier maps the cumulative behavioral profile of a user to a discrete tier and a continuous score in $[0, 100]$. The two tasks are mathematically distinct and implemented as separate inference paths within the same ML service, coupled in a single direction through one anomaly-derived feature consumed by the risk classifier (detailed below). The pipeline combines a three-algorithm supervised ensemble, a discrete-to-continuous score normalization, and a business rule override layer with a persistent audit trail.

Feature representation and classifiers. Each user is represented by a 9-dimensional feature vector (Table 7): three describe volume and amplitude (`total_transactions`, `mean_amount`, `std_amount`), two describe extremes (`max_amount`, `max_to_mean_ratio`), and four capture risk-relevant signals (`anomaly_rate`, `debt_ratio`, `spending_trend`, `amount_volatility`). Here `debt_ratio` denotes total expenses divided by total income, a proxy for over-spending rather than a balance-sheet liability measure. The `anomaly_rate` is itself a downstream aggregation of the anomaly detector's output, creating a directed dependency between the two inference paths. The risk classifier combines Random Forest (`n_estimators` = 200, `max_depth` = 8, `class_weight` = `balanced`), XGBoost (`n_estimators` = 200, `max_depth` = 4, `learning_rate` = 0.05), and Logistic Regression (`lbfgs` solver, `class_weight` = `balanced`, `StandardScaler` preprocessing). Tree depths are reduced relative to anomaly task to reflect the smaller training set (1,600 users vs 465,714 transactions).

#	Feature	Source	Bound / Default
1	total_transactions	Count of all transactions (Expense + Income)	—
2	mean_amount	Mean of expense amounts	—
3	std_amount	Std-dev of expense amounts	1.0 if undefined
4	max_amount	Maximum expense amount	—
5	anomaly_rate	Fraction of last 90 transactions flagged anomalous	—
6	debt_ratio	Total expense / total income	Clipped to [0, 2]; 1.0 if income = 0
7	spending_trend	Mean of last 30 expenses / mean of 31–90th expenses	Clipped to [0, 3]; 1.0 if history < 10
8	amount_volatility	std_amount / mean_amount	Clipped to [0, 5]
9	max_to_mean_ratio	max_amount / mean_amount	Clipped to [0, 50]

Table 7. Risk classification feature representation.

Ensemble combination and weight assignment. The three classifiers produce per-class posterior probability vectors over the three risk levels {Low, Medium, High}. The ensemble posterior is a weighted average:

$$P_{ensemble}(c | x) = w_{RF} \cdot P_{RF}(c | x) + w_{XGB} \cdot P_{XGB}(c | x) + w_{LR} \cdot P_{LR}(c | x)$$

with weights $(w_{RF}, w_{XGB}, w_{LR}) = (0.20, 0.70, 0.10)$ selected from a grid search over five candidate profiles; comparative results are reported in Section 5.1. XGBoost dominates with 0.70 as the strongest individual classifier on the validation set, a finding consistent with Sinaga and Machfud (2025), who report XGBoost outperforming Random Forest on accuracy, F1, and ROC-AUC for credit risk classification on imbalanced datasets. Logistic Regression receives the smallest weight of 0.10, reflecting the consistent finding in the credit risk literature (Odunlami & Nwonu, 2025) that gradient-boosted ensembles outperform linear classifiers on nonlinearly structured tabular features; its role is therefore retained as a probability-calibration anchor rather than as a primary discriminative component.

Continuous risk score normalization. Although the ensemble's argmax output identifies a discrete predicted class, AFRATS additionally produces a continuous score in the

range [0, 100] to enable gradient-based downstream prioritization. The transformation maps each predicted class to a base anchor — Low to 20.0, Medium to 55.0, High to 85.0 — and refines the score within the neighborhood of the anchor by incorporating the ensemble's probability mass on the High class:

$$risk_score = base_anchor + (P_{ensemble}(High) - 0.5) \times 20$$

clipped to [0, 100]. The final risk level emitted by the system is derived from the continuous score through two thresholds ($< 40 \rightarrow$ Low, $[40, 70) \rightarrow$ Medium, $\geq 70 \rightarrow$ High) rather than from the ensemble's argmax. This separation makes the final categorical output consistent with the continuous score, including after any business rule override has adjusted the score upward. The continuous output also enables thresholds to be retuned at deployment time without retraining — an operational property of the AFRATS design rather than a recommendation attributable to prior work.

Business rule override layer. Layered on top of the learned ensemble is an explicit business rule override mechanism that adjusts the continuous risk score under specific behavioral conditions not necessarily captured by the training distribution. Three rules are implemented, summarized in Table 8. The two debt-ratio rules are mutually exclusive and follow a hierarchical structure (most-restrictive wins) — a user with $debt_ratio = 1.6$ triggers Rule 1 only, not both. The anomaly boost is additive, applies after any debt rule, and is capped at 100. The overrides target the continuous score rather than the discrete class, which is then re-derived through the level thresholds described above. This architecture allows regulatory or operational expectations — for example, that a user spending substantially more than their income should not be classified below Medium risk regardless of the model's prediction — to coexist with the learned ensemble without retraining.

Rule	Trigger	Action
debt_high	$debt_ratio \geq 1.5$ and current score < 90	Floor score to 90.0
debt_medium	$debt_ratio \geq 1.0$ and current score < 60	Floor score to 60.0
anomaly_boost	$anomaly_rate \geq 0.3$	Add 15.0 to score (clipped to 100)

Table 8. Business rule override layer.

Audit trail and evaluation modes. Rather than blending model and rule outputs, AFRATS persists an audit trail in the `override_reasons` array — for example, “`debt_medium: debt_ratio=1.12>=1.0 → score floored to 60.0 (model: 50.46)`” — stored in the `RiskScores.Factors` JSON column alongside the underlying behavioral signals. The array is included only when an override produces an effective change in the score, supporting two distinct evaluation modes discussed in Section 5.2.

Production alignment. The feature engineering function, ensemble logic, score normalization, override layer, and audit trail are implemented in `app/ml/risk_models.py` and reused without modification by training and live inference. The three classifiers, scaler, label encoder, and ensemble weights are serialized into `models/risk_model.pkl` loaded at startup. A rule-based fallback preserves service continuity if the bundle is absent or any model invocation raises.

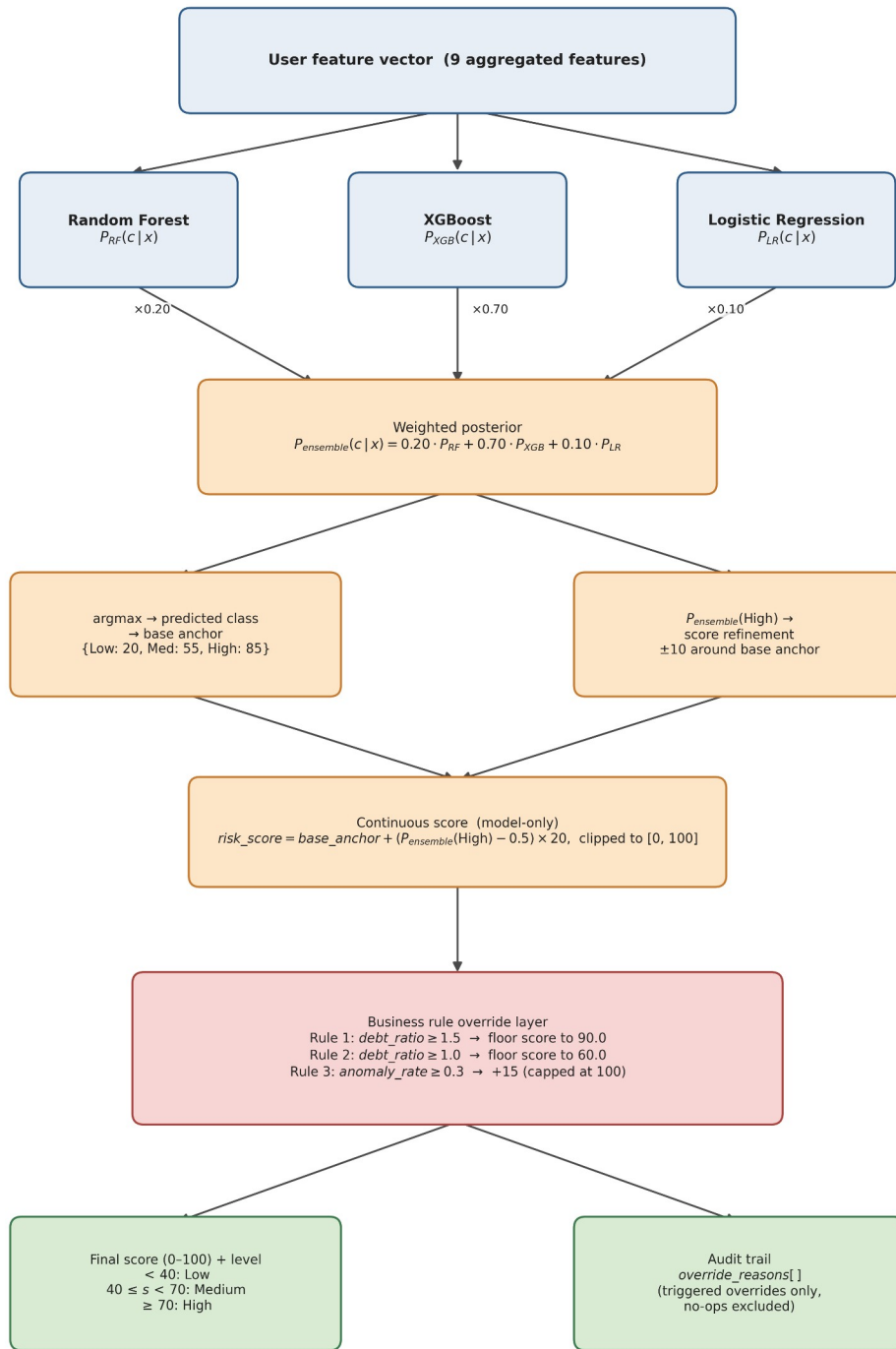


Figure 4. Risk classification decision flow.

3.4 System Architecture

The detection components do not operate in isolation — they are embedded in a distributed backend handling authentication, transaction ingestion, asynchronous event distribution, and notification delivery. Pattern-level discussion (microservices, Clean Architecture, API gateway authentication, event-driven communication) is developed in Chapter 2 and not repeated here.

Service decomposition and stack split. The system is partitioned into five independently deployable services along boundaries determined by structural concerns: AuthService isolates identity and credential handling; TransactionService owns the transactional ledger; MLService isolates the Python ecosystem and CPU-intensive inference; NotificationService isolates outbound delivery; and the API Gateway terminates all external traffic. The decomposition reflects differing scaling profiles, retraining cadences, and security boundaries rather than uniform feature partitioning. The four backend services run on ASP.NET Core 10; the inference layer runs on Python 3.11 with FastAPI. The ML ecosystem is most natively expressed in Python, while the transactional core benefits from .NET’s type system, dependency injection, and Entity Framework (EF) Core tooling. Microservice decomposition accommodates cross-language deployment without architectural penalty.

Communication topology. Inter-service communication splits synchronous and asynchronous channels by semantic. Synchronous HyperText Transfer Protocol (HTTP) through the Gateway handles query operations and writes whose response the caller must act on immediately. Asynchronous RabbitMQ messages handle side-effecting events: transaction.created from TransactionService to MLService, and analysis.completed from MLService to NotificationService. The Gateway terminates external traffic and propagates the authenticated identity to internal services — consistent with de Almeida and Canedo (2022). Figure 5 summarizes the topology.

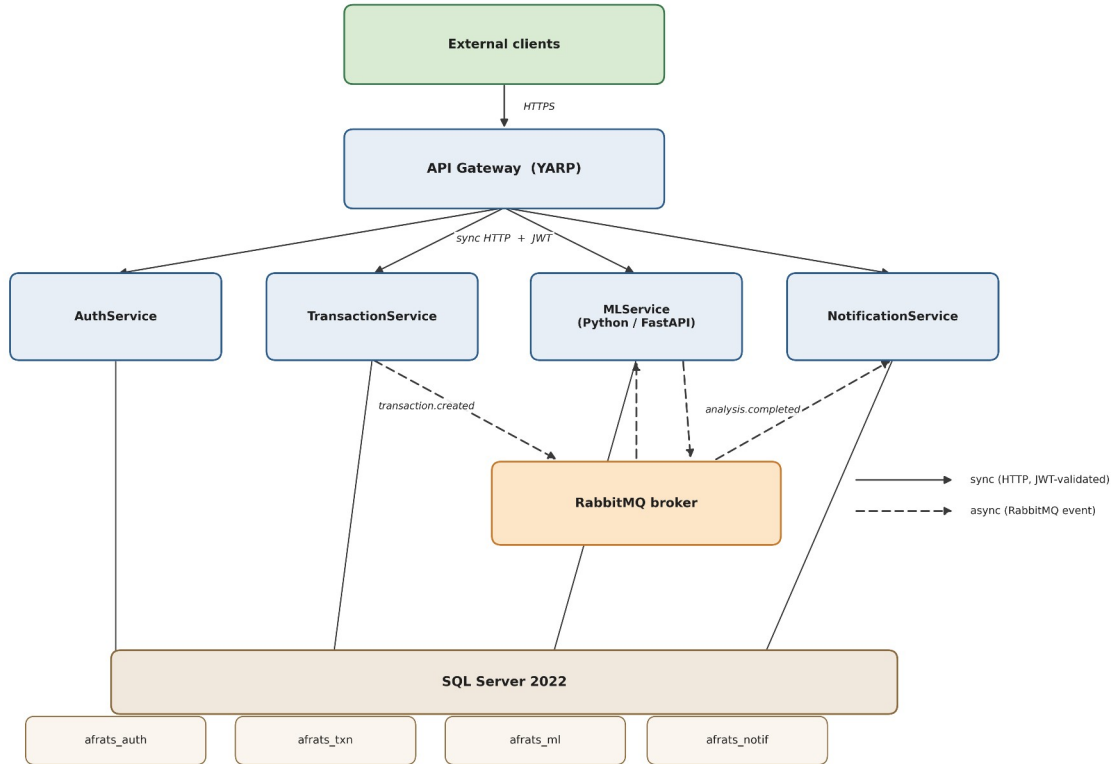


Figure 5. AFRATS service topology

3.5 Data Model

The persistence layer mirrors the service decomposition of Section 3.4: each service owns its own database within a shared Structured Query Language (SQL) Server instance ... rather than cross-database foreign keys. Per-service table inventories are detailed in Chapter 4. This section focuses on the two tables whose structure encodes a methodological commitment: *AnomalyLogs* and *RiskScores*.

AnomalyLogs structure. The *AnomalyLogs* table records anomaly detection results with a deliberately denormalized structure: for each analyzed transaction, five rows are persisted — one per algorithm (Z-score, Isolation Forest, LOF, XGBoost) recording its raw and normalized score, plus one final row recording the weighted ensemble score and verdict. Both the model-only score and the post-classification decision are stored on every row, allowing the same table to be queried under either a model-only view or a post-decision view — directly supporting the disentanglement modes used in Section 5.2.

RiskScores and Factors JSON. The *RiskScores* table stores the current risk state for each user — one row per user, updated rather than appended. Each row contains the user

identifier, continuous score in [0, 100], discrete risk level, a CalculatedAt timestamp, and a Factors column typed as JSON. The Factors object always carries the underlying behavioral signals (anomaly_weight, debt_ratio, spending_trend) — where anomaly_weight is the model-level anomaly_rate scaled to 0–100 for display ($\text{anomaly_rate} \times 100$) — and additionally carries an override_reasons array if and only if at least one business rule override produced an effective change in the score. Figure 6 shows a representative payload.

```
{
  "anomaly_weight": 12.0,
  "debt_ratio": 1.2871,
  "spending_trend": 1.0,
  "override_reasons": [
    "debt_medium: debt_ratio=1.29 >= 1.0 → score floored to 60.0 (model: 51.29)"
  ]
}
```

Figure 6. RiskScores.Factors JSON payload (rule-influenced score).

Cross-database identity. Although no foreign key constraint spans databases, every user is identified by the same UserId Globally Unique Identifier across afrats_auth, afrats_txn, afrats_ml, and afrats_notif. Identity propagation is maintained at the application layer — the Gateway extracts UserId from the validated JWT and forwards it; asynchronous event payloads carry UserId and TransactionId alongside domain data. The trade-off is explicit: cross-database referential integrity is sacrificed for independent schema evolution, with consistency enforced by event-payload contracts. Figure 7 summarizes the relevant tables, primary keys, and identity columns crossing the asynchronous boundary.

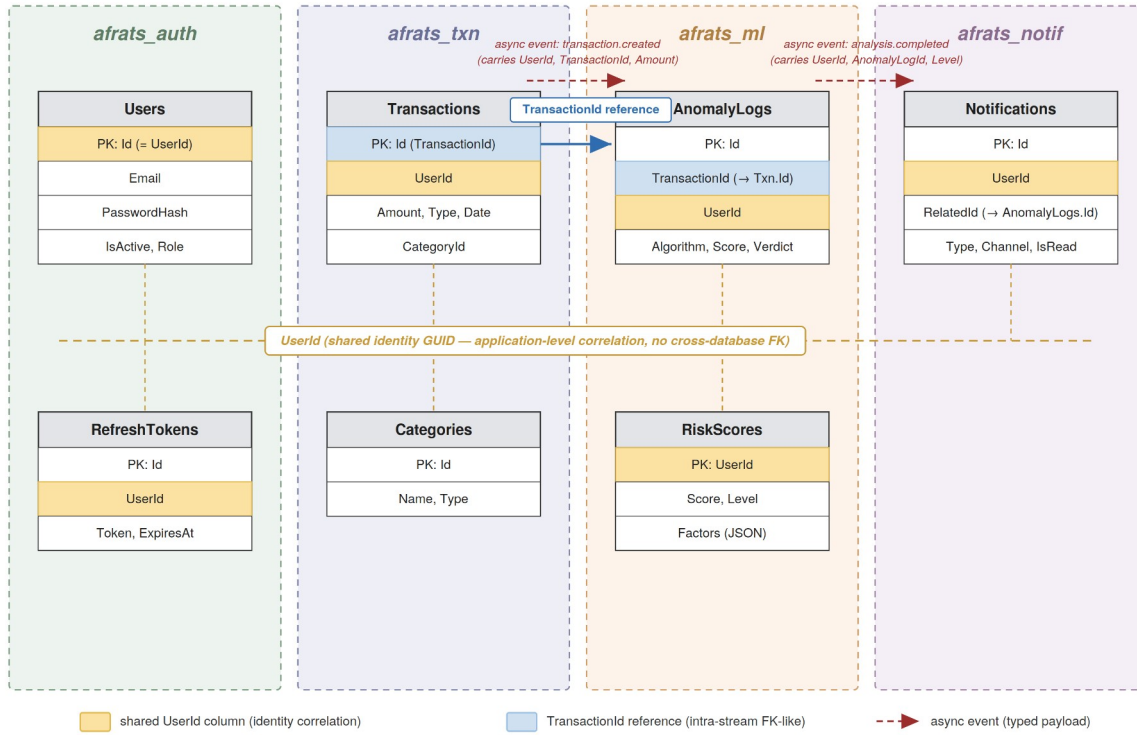


Figure 7. Cross-database entity relationships

3.6 Event-Driven Communication Design

The asynchronous flow connecting `TransactionService`, `MLService`, and `NotificationService` is mediated by RabbitMQ 3.13. This section describes the broker topology, event sequence, and mechanisms guaranteeing correct behavior under at-least-once delivery.

Broker topology. Two topic exchanges and three durable queues (Table 9) separate transaction-side publication (`afrats.transactions`, by `TransactionService`) from inference-side publication (`afrats.ml`, by `MLService`). All three queues are durable, surviving broker restarts. Each queue binds to a single consumer rather than fanning out, simplifying message ordering reasoning within a queue. Three operational properties shape behavior: persistent message delivery (disk-written before publisher acknowledgment), per-service prefetch (`MLService` uses `prefetch_count = 1` for `AnomalyLogs` idempotency safety; `NotificationService` uses `prefetch_count = 10` since email latency would otherwise dominate throughput), and manual acknowledgment mode (removing messages only after explicit confirmation).

Exchange	Queue	Routing Key	Consumer
afrats.transactions	ml.transaction.created	transaction.created	MLService
afrats.ml	txn.analysis.completed	analysis.completed	TransactionService
afrats.ml	notif.high.risk.detected	high.risk.detected	NotificationService

Table 9. RabbitMQ broker topology.

Event flow. When a user creates a transaction, TransactionService persists it and publishes transaction.created to afrats.transactions with the transaction details and a userHistory enrichment carrying the user’s most recent transactions. This enrichment makes MLService stateless with respect to historical data and eliminates cross-service queries. MLService consumes the event, applies the anomaly and risk pipelines (Sections 3.2–3.3) including overrides, persists AnomalyLogs and RiskScores rows, and publishes analysis.completed carrying verdict, scores, and model version. If the risk score exceeds 70, MLService additionally publishes high.risk.detected carrying the new and previous scores plus the Factors JSON with override_reasons preserved. TransactionService consumes analysis.completed and updates IsAnomalous and AnomalyScore on the originating transaction; NotificationService consumes analysis.completed for in-app notifications and high.risk.detected for high-risk alerts, delivering in-app and email notifications subject to per-user preferences. Figure 8 presents the end-to-end sequence.

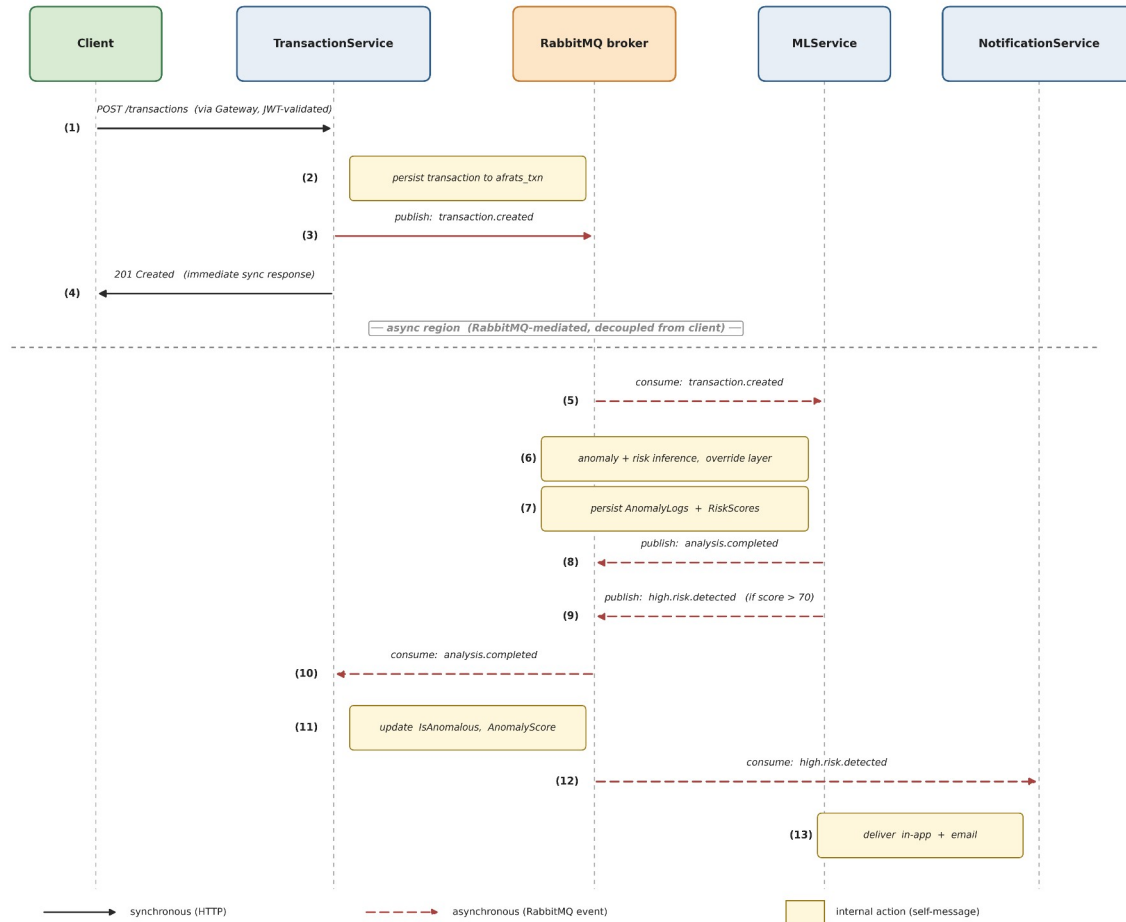


Figure 8. End-to-end event flow sequence.

Idempotency and error handling. AMQP at-least-once delivery can produce duplicate event reception. AFRATS protects state through three layered checks: unique message identifiers preserved across redeliveries; consumer-side state inspection in MLService (an AnomalyLogs lookup before re-running inference); and downstream idempotency — TransactionService uses update rather than insert semantics, and NotificationService deduplicates on (UserId, RelatedId, Type, Channel) before persisting. Consumers distinguish transient errors (requeue = true for retry) from permanent errors (requeue = false, logged for operator review); a dead-letter exchange is acknowledged as a hardening direction in Section 6.2.

Chapter 4

Implementation

This chapter describes the implementation of AFRATS, covering the software stack and cross-cutting patterns, the individual backend services, container orchestration, and the client-side application.

4.1 Software Stack and Cross-Cutting Patterns

AFRATS is implemented on a deliberately mixed-language stack. The four backend services (AuthService, TransactionService, NotificationService, API Gateway) are built on ASP.NET Core 10; MLService is built on Python 3.11 with FastAPI. The architectural rationale is developed in Section 3.4. The shared library surface realizes Clean Architecture across both stacks: the .NET services use MediatR for command and query dispatch, FluentValidation as a pipeline behavior, AutoMapper for entity-to-Data Transfer Object (DTO) mapping, Entity Framework Core 10 against SQL Server 2022, and Serilog for structured logs. MLService uses FastAPI with Uvicorn, NumPy and pandas, scikit-learn and xgboost, SQLAlchemy with pyodbc, and pika for RabbitMQ consumption. Trained models are serialized with joblib and loaded once at startup. Table 10 summarizes the layer mapping.

Clean Architecture Layer	.NET Services (ASP.NET Core 10)	MLService (Python 3.11 + FastAPI)
API	API/ — controllers, middleware	routers/ (FastAPI endpoints), schemas/ (Pydantic DTOs)
Application	Application/ — MediatR handlers, FluentValidation validators, AutoMapper profiles	services/ (orchestration)
Domain	Domain/ — entities, value objects	ml/ (model code, feature engineering, model manager)
Infrastructure	Infrastructure/ — EF Core 10, RabbitMQ publishers/consumers	db/ + db/repositories/ (SQLAlchemy), messaging/ (pika)
Cross-cutting	Distributed via middleware (Serilog, exception handlers, JWT)	core/ — config, security, exceptions

Table 10. Implementation libraries by Clean Architecture layer.

CQRS and JWT contract. Write operations are MediatR commands, read operations are queries; FluentValidation pipeline behaviors run uniformly before each handler. Domain exceptions are translated to HTTP status codes by exception-handling middleware, preserving the architectural rule that HTTP knowledge remains outside business logic. All four .NET services configure JwtBearer with Hash-based Message Authentication Code (HMAC)-SHA256 symmetric signing through environment-bound configuration sections — Secret, Issuer, and Audience values used by AuthService at issuance time are identical to those used by all downstream services at validation. MLService’s FastAPI authentication dependency validates against the same Secret, guaranteeing JWT validation is equivalent across implementation languages.

4.2 AuthService

AuthService is responsible for the entire authentication lifecycle: user registration, login, refresh token rotation, password management, and email confirmation. It is the only service that stores password hashes and issues JWT access tokens. Passwords are hashed using Blowfish-based cryptographic hash function (BCrypt) with work factor 12 (250–300 ms per hash on a contemporary CPU), following Open Worldwide Application Security Project recommendation. JWT access tokens carry user identifier, email, role, jti, and timestamps, with a 15-minute lifetime limiting the operational value of a compromised token. Refresh token rotation follows OAuth 2.0 best current practice (Request for Comments (RFC) 9700): each refresh marks the supplied token as revoked and issues a new pair; any re-use of a revoked token is treated as a replay attack and triggers revocation of the entire refresh family, forcing re-authentication.

4.3 TransactionService

TransactionService implements the financial domain model — transactions, categories, and the read-side history queries on which downstream consumers depend. It is the only service writing the Transactions and Categories tables. When the CreateTransactionCommandHandler commits a transaction, it publishes transaction.created to RabbitMQ as the final step, wrapped in a try-catch that logs failures without rolling back: a publication failure produces a transaction that exists in the database but never reaches MLService — a degraded but not corrupted state.

A stricter transactional outbox is acknowledged as a hardening direction in Section 6.2. The Transactions and Categories tables implement soft deletion through an IsDeleted column with an EF Core global query filter at the DbContext level, eliminating an entire class of “forgot to filter” bugs.

4.4 MLService

MLService is the dedicated inference layer described methodologically in Sections 3.2–3.3. At startup, MLService loads two pre-trained model bundles: models/anomaly_model.pkl containing Isolation Forest, LOF, LOF scaler, and XGBoost; and models/risk_model.pkl containing Random Forest, XGBoost, Logistic Regression, feature scaler, label encoder, and ensemble weights. Both are serialized with joblib and carry version strings, feature column lists, and metadata to verify schema compatibility. Models are loaded once and held as singleton dependencies. The feature engineering code used at inference is the same code used at training — both defined in app/ml/feature_engineering.py and imported by both training scripts and the inference pipeline. Sharing the code path eliminates the silent regression risk of divergent feature definitions. Synchronous HTTP endpoints (health checks, on-demand inference) validate bearer tokens against the shared Secret with a user-identifier match guard preventing inference on another user’s data.

4.5 Notification

Two independent RabbitMQ consumers run: AnalysisCompletedConsumer (notif.analysis.completed → in-app notifications for flagged transactions) and HighRiskDetectedConsumer (notif.high.risk.detected → in-app and email via MailKit/Simple Mail Transfer Protocol (SMTP) when risk score crosses the high-risk threshold of 70). Each maintains its own acknowledgment state. When a notification is generated through multiple channels, the service persists one Notifications row per channel: a high-risk event therefore writes two rows sharing RelatedId but distinguished by Channel value. The unread count query filters on Channel = InApp AND IsRead = false, naturally excluding email from the unread badge; channel failures are independent. Application-level idempotency uses a (UserId, RelatedId, Type, Channel) tuple lookup before each insert.

4.6 Gateway Layer

The Gateway is a YARP reverse proxy on ASP.NET Core 10. All client traffic enters here; no downstream service is reachable from outside the internal container network. YARP routes are loaded from `appsettings.json`; destination clusters resolve to internal container hostnames such as `http://afrats-auth:8080` and `http://afrats-ml:8000`. The authorization configuration applies a `FallbackPolicy` requiring authenticated users for any route without an explicit “anonymous” opt-out — a developer adding a new route who forgets to specify a policy receives JWT-required behavior automatically rather than inadvertently exposing the endpoint. JWT validation at the edge plus enforcement at each downstream service provides defense in depth. Cross-Origin Resource Sharing (CORS) uses a strict origin allowlist with restricted methods, headers, and preflight cache duration.

4.7 Container Orchestration

The deployment is orchestrated through Docker Compose with seven containers: the four .NET services, `MLService`, `SQL Server 2022`, and `RabbitMQ 3.13`. Each application service is built from a multi-stage Dockerfile — a build stage compiles or installs dependencies, followed by a slim runtime stage copying only what is needed to run the service — keeping the runtime image small. Startup is ordered through `depends_on` declarations combined with healthchecks: infrastructure containers expose health probes, application services wait on `service_healthy`, and the Gateway starts last so all routes are reachable as soon as the externally exposed port becomes responsive. All containers share a single bridge network with container-name Domain Name System resolution; only the Gateway port is published to the host. Environment-varying configuration (connection strings, JWT Secret/Issuer/Audience, broker credentials, SMTP, CORS) is supplied through `.env` files following twelve-factor principles; named volumes preserve SQL Server data and RabbitMQ queue state. The complete container inventory is summarized in Table 11.

Container	Image	Port	Depends On	Volume
afrats-auth	aspnet:10.0	8080	sql	—
afrats-transaction	aspnet:10.0	8080	sql, rabbitmq	—
afrats-ml	python:3.11-slim	8000	sql, rabbitmq	./models (ro)
afrats-notification	aspnet:10.0	8080	sql, rabbitmq	—
afrats-gateway	aspnet:10.0	8080 → 5000	all app services	—
afrats-sql	mssql/server:2022	1433	—	mssql_data
afrats-rabbitmq	rabbitmq:3.13-mgmt	5672, 15672	—	rabbitmq_data

Table 11. AFRATS container inventory and orchestration metadata.

4.8 Frontend Application

The user-facing application is a single-page web application built with React, Vite, and Tailwind CSS. A React Native mobile companion shares the same backend API and consumes the same Gateway endpoints. Cross-platform consistency is maintained by reusing shared TypeScript types, request schemas, and API client logic; Figures 9 and 10 illustrate the Dashboard view on the web and mobile clients respectively — the primary user-facing surface that aggregates anomaly alerts, recent transactions, and the current risk score. Drill-down views (anomaly detail, risk profile, transaction list) follow the same component vocabulary and are omitted for brevity. Client-side state is managed through React's context and reducer primitives; routing through react-router-dom. Authentication state stores the JWT access token in memory and the refresh token in a secure cookie with automatic refresh. A shared apiClient module handles JWT injection and error response normalization across the two distinct error contracts emitted by AuthService (RFC 9110 ProblemDetails) and TransactionService (custom envelope shape).

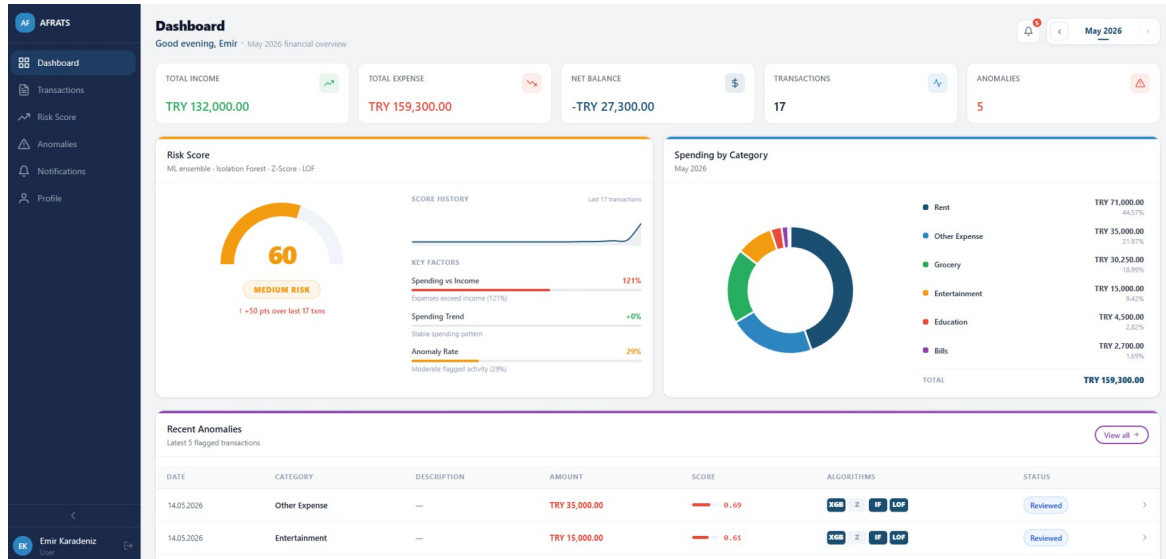


Figure 9. Dashboard view on the web client.

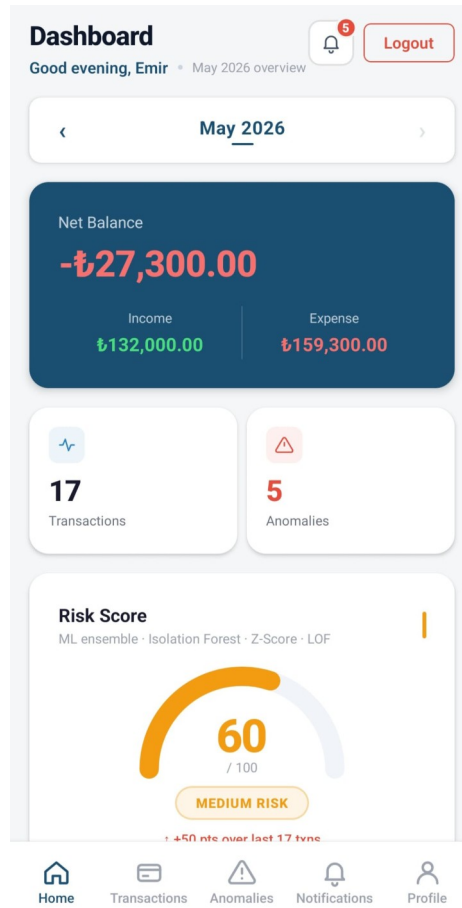


Figure 10. Dashboard view on the React Native mobile client.

Chapter 5

Evaluation and Results

This chapter reports the experimental results for both inference paths and the end-to-end system, then discusses their implications.

5.1 Results

This section reports the empirical results obtained for both inference paths of AFRATS and the end-to-end system validation. Evaluation followed the methodology defined in Chapter 3: a fixed train/test split with seed = 42, user-level stratified sampling preventing cross-user information leakage, and explicit reporting of both noisy-label and clean-label regimes for the supervised risk classifier. The results below cover anomaly detection, risk classification, and end-to-end system validation including latency characterization.

The anomaly detection ensemble was evaluated on 98,536 test-set transactions belonging to 300 test users, with an empirical anomaly rate of 4.69% — closely matching the 4.9% contamination parameter used during training (Section 3.2).

Per-algorithm baselines. Table 12 reports six metrics — F1 score, precision, recall, MCC, ROC-AUC, and the area under the precision–recall curve (PR-AUC) — for each of the four detectors evaluated independently. The supervised XGBoost detector dominates all unsupervised alternatives, with F1 = 0.765 against the next-best Z-score at 0.364, reflecting the structural advantage of supervised learning on a labeled detection task. The three unsupervised detectors all fall below 0.37 individually, supporting their inclusion at lower weights as carriers of complementary signal. Figure 11 visualizes the per-algorithm F1 and MCC comparison.

Detector	F1	Precision	Recall	MCC	ROC-AUC	PR-AUC
Z-score	0.364	0.419	0.322	0.340	0.879	0.449
Isolation Forest	0.355	0.331	0.385	0.322	0.920	0.320
Local Outlier Factor	0.304	0.248	0.393	0.269	0.778	0.219
XGBoost	0.765	0.637	0.957	0.768	0.996	0.939

Table 12. Per-algorithm metrics on the test set.

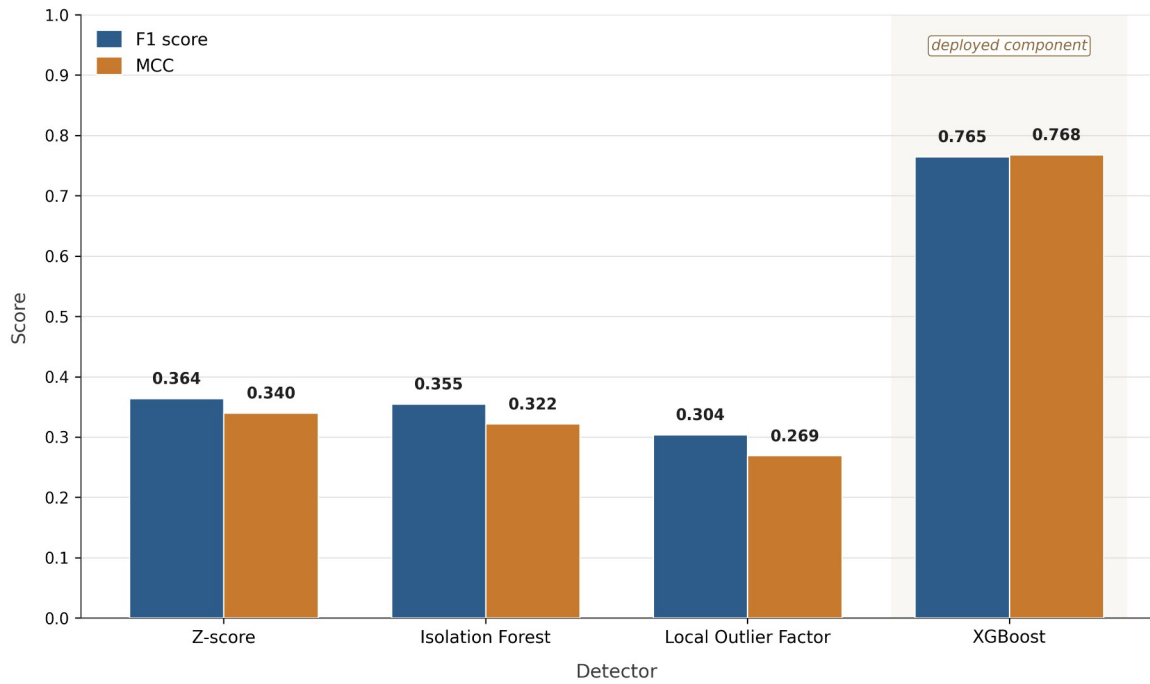


Figure 11. Per-algorithm F1 and MCC comparison

Ensemble strategy comparison. Building on these per-algorithm baselines, three ensemble strategy families were evaluated, and Table 13 summarizes their performance. Strict conjunction proves overly conservative at $F1 = 0.316$, majority vote loses precision benefits at $F1 = 0.544$, and the weighted-score family outperforms both — motivating fine-grained weight profile calibration within this family.

Strategy	Weights (Z / IF / XGB / LOF)	F1	Precision	Recall	MCC
Strict conjunction ($Z \wedge IF$)	—	0.316	0.638	0.210	0.350
Majority vote (≥ 2 of 4)	—	0.544	0.516	0.575	0.521
Weighted — Default	0.40 / 0.25 / 0.30 / 0.05	0.700	0.798	0.624	0.693
Weighted — IF-heavy	0.30 / 0.40 / 0.25 / 0.05	0.690	0.771	0.624	0.681
Weighted — Balanced	0.30 / 0.30 / 0.35 / 0.05	0.801	0.776	0.828	0.791
Weighted — XGB-heavy (deployed)	0.30 / 0.20 / 0.45 / 0.05	0.813	0.733	0.914	0.809

Table 13. Ensemble strategies and weight profile metrics.

Weight profile selection and confusion analysis. Four weight profiles were evaluated within the weighted score family; the deployed XGB-heavy profile (0.30 / 0.20 / 0.45 / 0.05) achieves $F1 = 0.813$, precision = 0.733, recall = 0.914, MCC = 0.809, ROC-AUC = 0.993, PR-AUC = 0.895. The ensemble F1 of 0.813 exceeds standalone XGBoost’s 0.765 — a +0.048 lift confirming that unsupervised detectors contribute marginal value beyond the supervised signal alone (Figure 11). For the deployed profile, the test set produces 4,223 true positives, 92,375 true negatives, 1,542 false positives, and 396 false negatives — miss rate 8.6%, false alarm rate 1.6% (Figure 12). The recall-leaning posture (91.4% recall against 73.3% precision) is operationally deliberate: a missed anomaly forwards undetected harm, whereas a false alarm produces a reviewable notification.

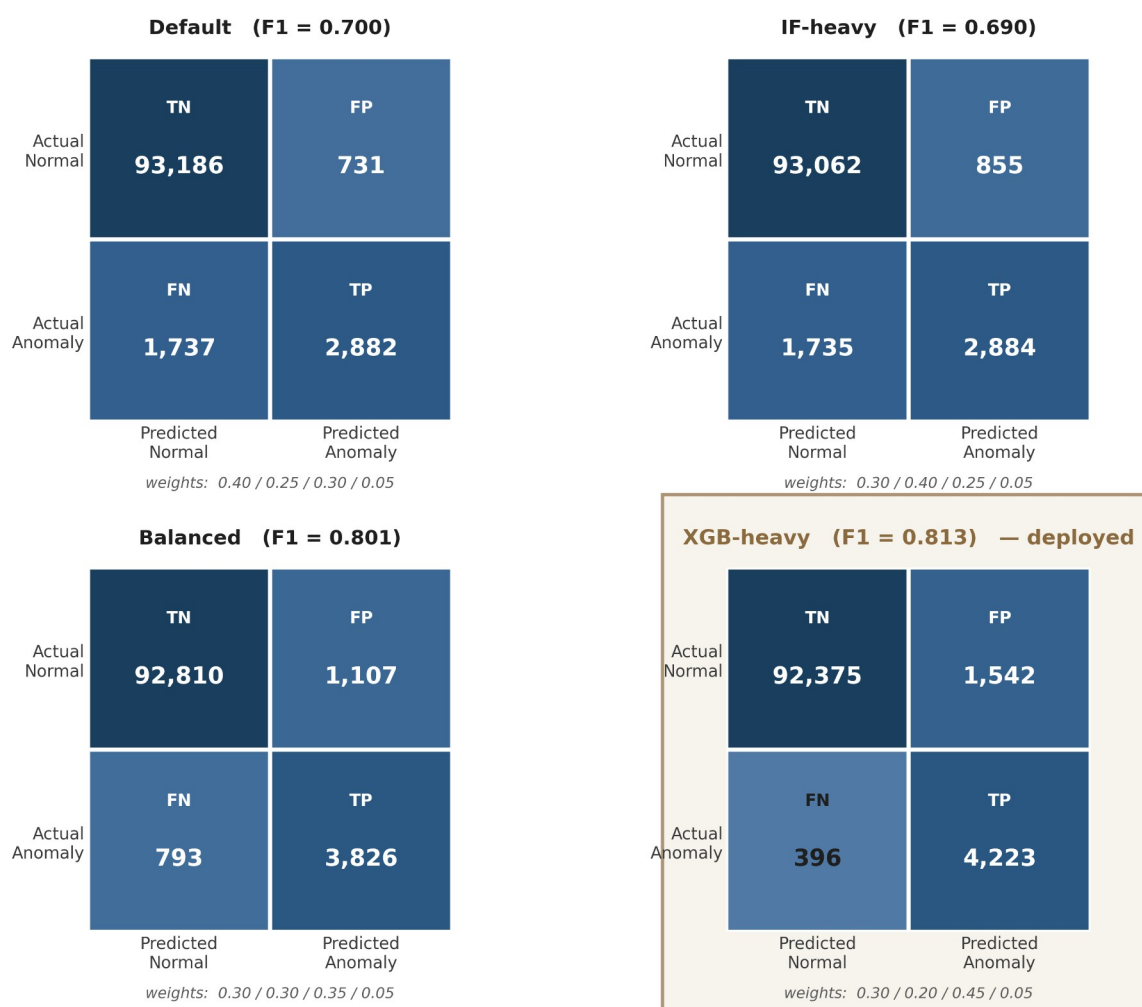


Figure 12. Confusion matrices for the four weight profiles

Turning to the user-level inference path, the risk classification ensemble was evaluated on a held-out test set of 400 users stratified from the 2,000-user dataset. The classifier is trained

on the noisy labels (7% deliberately flipped) and evaluated under two complementary regimes: noisy-label (production-realistic) and clean-label (isolating learning capacity).

Per-classifier baselines. Table 14 reports accuracy, macro F1, and MCC for each classifier independently under the noisy-label regime. XGBoost leads with accuracy 0.770 and macro F1 0.718, against Random Forest at 0.718 / 0.656 and Logistic Regression at 0.725 / 0.669. The smaller gap relative to anomaly detection justifies Logistic Regression's retention as a calibration anchor rather than as a dominant predictor.

Classifier	Accuracy	Macro F1	MCC
Random Forest	0.718	0.656	0.500
XGBoost	0.770	0.718	0.582
Logistic Regression	0.725	0.669	0.526

Table 14. Per-classifier metrics on the noisy-label test set.

Ensemble calibration and deployed profile. Five weight profiles were evaluated, and Table 15 summarizes their performance under the noisy-label regime. Performance improves monotonically as XGBoost weight increases, with the deployed XGB-dom profile (RF 0.20 / XGB 0.70 / LR 0.10) reaching accuracy 0.768, macro F1 0.714, MCC 0.580 — effectively tying, and on accuracy nominally trailing, XGBoost alone (0.770) within evaluation noise. This is reported honestly: ensemble retention is justified by better-calibrated posterior probabilities and the architectural property of being reweightable at deployment without retraining, not by raw metric improvement.

Profile	Weights (RF / XGB / LR)	Accuracy	Macro F1	MCC
Default	0.45 / 0.45 / 0.10	0.753	0.694	0.556
RF-heavy	0.55 / 0.35 / 0.10	0.753	0.692	0.557
Balanced	0.40 / 0.50 / 0.10	0.755	0.698	0.560
XGB-heavy	0.30 / 0.60 / 0.10	0.763	0.706	0.572
XGB-dom (deployed)	0.20 / 0.70 / 0.10	0.768	0.714	0.580

Table 15. Ensemble weight profile metrics on the noisy-label test set.

Clean-label evaluation and confusion analysis. Reporting only noisy-label metrics would conflate failure to learn versus the 7% noise floor. Evaluated against the retained clean ground-truth labels, the deployed ensemble achieves accuracy 0.8125, macro F1 0.761, MCC 0.656 — substantially above the noisy-label numbers. The 4.5-percentage-point gap is consistent with the 6.75% of test users (27/400) flipped during noise injection, indicating the gap is attributable to label corruption rather than learning failure. Figure 13 shows the corresponding clean-label confusion matrix (diagonal 325/400, accuracy 0.8125): Low is classified reliably (209/239, none misclassified as High); Medium reaches 91/123, with its errors split between Low and High; High recovers to 25/38. The Medium-High confusion reflects deliberate class-boundary overlap built into the synthetic dataset (Section 3.1).

		<i>Predicted class</i>		
		Low	Medium	High
<i>Actual class</i>	Low	209 87.4% of actual Low	30 12.6% of actual Low	0 0.0% of actual Low
	Medium	22 17.9% of actual Medium	91 74.0% of actual Medium	10 8.1% of actual Medium
	High	0 0.0% of actual High	13 34.2% of actual High	25 65.8% of actual High

n = 239
n = 123
n = 38

Test set: *N* = 400 users (clean-label evaluation)

Figure 13. Confusion matrix for the deployed ensemble (clean-label evaluation)

Feature importance. Figure 14 reports Random Forest Gini importance. The most influential feature is `debt_ratio` at 0.389 — more than twice `anomaly_rate` at 0.159. Convergence between the learned model and rule layer on the same feature (`debt_ratio` is also the primary override trigger of Section 3.3) provides independent confirmation that `debt_ratio` carries the strongest risk signal in this dataset; the result is dataset-dependent and should not be transferred to datasets calibrated to different consumption dynamics.

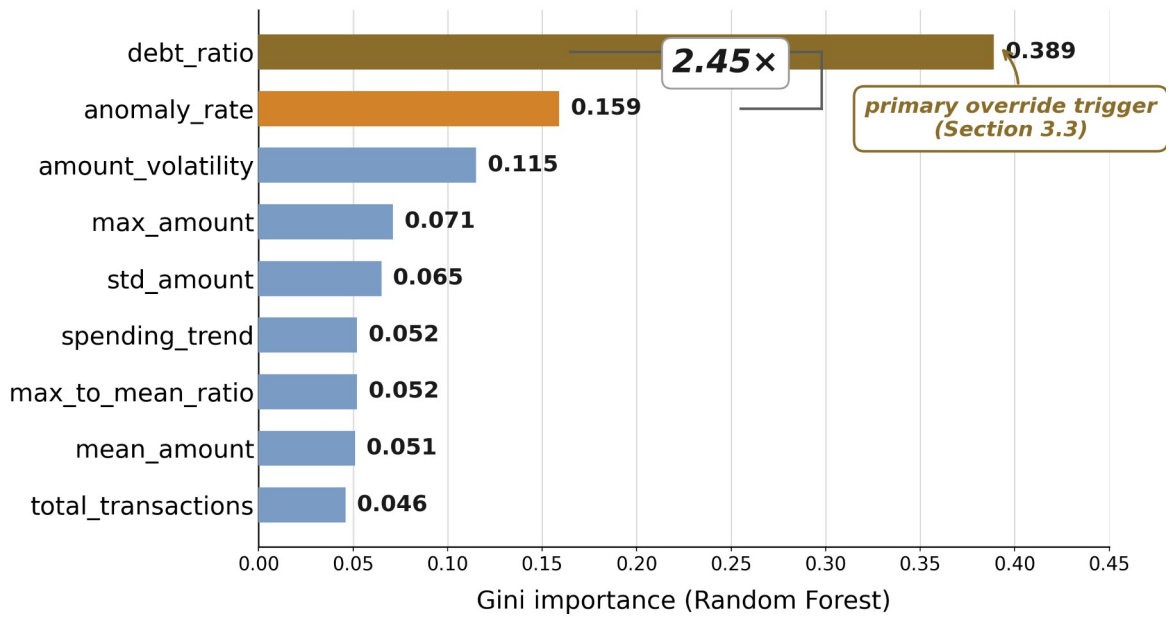


Figure 14. Random Forest Gini feature importance

The metrics reported above establish the performance of the two inference paths in isolation. A final end-to-end smoke test exercises the full event flow against a containerized deployment, combined with latency characterization across ten sequential transactions.

Methodology and illustrative transactions. The smoke test was conducted against the fully containerized AFRATS deployment with all five services running concurrently alongside a live RabbitMQ broker and SQL Server 2022. Two transactions exercised the principal decision paths. Transaction A was a 25,000 TRY Grocery expense, approximately 2.7 standard deviations above the user's historical mean. The Gateway validated the JWT and forwarded to TransactionService, which committed and returned 201 Created. Asynchronously, TransactionService published transaction.created; MLService consumed, extracted the 12-feature representation, and applied the ensemble. Per-algorithm scores $Z = 0.5324$, $IF = 0.5624$, $XGB = 1.0000$, $LOF = 0.8188$ recorded in AnomalyLogs reproduce the ensemble score as $0.30 \cdot 0.5324 + 0.20 \cdot 0.5624 + 0.45 \cdot 1.0000 + 0.05 \cdot 0.8188 = 0.7631$, exceeding the 0.5 threshold; correctly flagged. Transaction B was an 850 TRY Grocery expense, within one standard deviation of the user's mean. The ensemble score was 0.125, correctly not flagged. However, the user's debt_ratio had reached 1.12, triggering debt_medium. The override floored the model-only risk score of 50.46 to 60.0, producing the override_reasons entry: "debt_medium: debt_ratio=1.12>=1.0 → score floored to 60.0 (model: 50.46)" — realizing the audit-trail design of Section 3.3.

Latency characterization. Ten transactions submitted with one-second spacing were measured on two intervals (Table 16). Synchronous response (POST → 201 Created) averages 31 ms, dominated by Gateway routing, JWT validation, and transaction persistence. End-to-end (POST → AnomalyLogs externally visible) averages 252 ms, the additional time covering RabbitMQ delivery, four-detector inference, and five-row AnomalyLogs persistence — all completing under 300 ms in every observation.

Stage	Mean (ms)	Std (ms)	Min (ms)	Max (ms)
Synchronous response (POST → 201 Created)	30.8	12.1	14.5	52.9
End-to-end (POST → AnomalyLogs externally visible)	251.8	15.7	227.5	277.1

Table 16. End-to-end latency statistics

Production artifacts and validation summary. Live operational evidence was collected by querying RiskScores directly against the running database. Figure 15 shows ten consecutive updates for a single user, the Factors JSON column, and the override_reasons audit array — matching the design in Figure 6 exactly, with both debt_high and debt_medium rules visible firing on different user states. Together with latency characterization, the test exercises four key system properties: synchronous response path, asynchronous event-driven inference, business rule override with persistent audit trail, and end-to-end timing. All four operated as specified, with no inconsistencies between computed metrics and persisted state.

	CalculatedAt	RiskScore	RiskLevel	DebtRatio	OverrideReasons
1	2026-05-17 21:22:37	90.00	High	2.0	["debt_high: debt_ratio=2.00>=1.5 \u2192 score floored to 90.0 (model: 87.31)"]
2	2026-05-17 21:22:36	90.00	High	2.0	["debt_high: debt_ratio=2.00>=1.5 \u2192 score floored to 90.0 (model: 86.46)"]
3	2026-05-17 21:22:34	90.00	High	1.9778	["debt_high: debt_ratio=1.98>=1.5 \u2192 score floored to 90.0 (model: 86.83)"]
4	2026-05-17 21:22:33	90.00	High	1.5509	["debt_high: debt_ratio=1.55>=1.5 \u2192 score floored to 90.0 (model: 53.00)"]
5	2026-05-17 21:22:32	60.00	Medium	1.1241	["debt_medium: debt_ratio=1.12>=1.0 \u2192 score floored to 60.0 (model: 50.46)"]
6	2026-05-17 21:22:30	47.05	Medium	0.8395	NULL
7	2026-05-17 21:22:29	11.59	Low	0.6023	NULL
8	2026-05-17 21:22:27	11.83	Low	0.5881	NULL
9	2026-05-17 21:12:16	11.69	Low	0.5739	NULL
10	2026-05-17 21:12:15	11.50	Low	0.5545	NULL

Figure 15. RiskScores rows for a single user

5.2 Discussion

This section interprets the methodological choices and experimental results against the literature reviewed in Chapter 2. Three themes structure the analysis: the implications of the algorithmic results for ensemble design philosophy, the value of the dual-regime evaluation methodology, and the operational significance of separating learned model decisions from business rule overrides.

The dominant finding is the consistent advantage of supervised gradient boosting across both detection tasks: XGBoost achieves $F1 = 0.765$ in anomaly detection (next-best Z-score at 0.364) and accuracy = 0.770 in risk classification, consistent with Sinaga and Machfud (2025) and Odunlami and Nwonu (2025). Whether the ensemble adds value on top of XGBoost is task-specific: in anomaly detection the ensemble achieves $F1 = 0.813$ against XGBoost-only at 0.765, a +0.048 lift confirming that structurally different unsupervised biases recover information any single detector discards; in risk classification the ensemble (macro $F1 = 0.714$) nominally trails XGBoost-only (0.718) within evaluation noise. The methodological lesson is that ensemble value must be measured rather than assumed. The dominance of `debt_ratio` in feature importance is itself dataset-dependent and should be read as a property of this evaluation rather than a transferable claim about credit risk modeling.

Beyond the algorithmic results, the clean-versus-noisy label evaluation illustrates a methodological property rarely available in the financial anomaly detection literature. Studies relying on public datasets (most commonly the Credit Card Fraud Detection Dataset) operate under the implicit assumption that published labels are correct, since no ground truth is available; classifier failure modes cannot be separated from labels that may themselves be incorrect. The synthetic dataset dissolves this conflation: by injecting 7% controlled noise and retaining clean labels, AFRATS evaluates the same classifier under both regimes, with the 4.5-percentage-point gap consistent with the test-set noise rate (6.75%) — supporting the interpretation that the gap reflects label corruption rather than learning failure. The synthetic construction additionally eliminates feature-label leakage: behavioral parameters are sampled from distributions rather than computed from rules, preventing a sufficiently expressive classifier from recovering the labeling rule. Combined with user-level stratified splitting, the dataset provides three structural guarantees public benchmarks typically do not.

Finally, the business rule override layer with persistent audit trail distinguishes AFRATS from the literature in Chapter 2. Most empirical evaluations report only model-level

metrics, treating rule-based logic running alongside the model as outside scope — methodologically convenient but operationally misleading, since end-to-end metrics in any production financial system are a function of both. AFRATS makes the dual influence visible: the smoke test demonstrated a model-only risk score of 50.46 floored to 60.0 by the `debt_medium` rule, with both values plus the textual override reason persisted. Post-hoc analytics can partition records by the presence or absence of `override_reasons` to produce two populations — a model-only population for clean classifier performance measurement, and a post-override population reflecting externally observable decisions. By making both answerable from the same persisted record, AFRATS treats the model-versus-rule boundary as an architectural decision rather than an analytical inconvenience.

Chapter 6

Conclusion and Future Directions

This chapter summarizes the main findings, acknowledges the principal limitations, and outlines directions for future work.

6.1 Conclusion

This thesis addressed two parallel challenges: single-algorithm ML detectors and static rule-based risk scoring cannot adapt to the diversity of anomalous behavior or the evolution of user profiles, while monolithic implementations couple detection logic to transaction processing in ways that constrain independent retraining and scaling. AFRATS responded through four integrated contributions: a five-service event-driven microservices backend, a four-algorithm anomaly detection ensemble ($F1 = 0.813$), a three-algorithm supervised risk classification ensemble (81.25% accuracy under clean-label evaluation), and a model-versus-rule disentanglement layer paired with a TÜİK-calibrated synthetic dataset. Three broader observations follow: supervised gradient boosting was the dominant single contributor across both tasks; dual-regime evaluation provided a methodological view rarely available in the literature, separating learning capacity from annotation noise; and persisting the override audit trail made the model-versus-rule boundary directly queryable. The end-to-end smoke test demonstrated these decisions integrate into a working system within the latency envelope of Table 16.

6.2 Limitations

Two limitations should be acknowledged. They constrain claims about deployment readiness and motivate Section 6.3.

Synthetic data and external validity. Both ensembles are trained and evaluated on a synthetic dataset calibrated to Turkish household consumption (Section 3.1). It was constructed to prevent feature-label leakage, cross-user contamination, and unbounded noise, but the reported metrics are not validated against real anonymized transaction streams. The reported results should be read as characterizing system behavior under the synthesis assumptions rather than production-deployment guarantees.

Operational hardening is partial. Several patterns standard in production financial systems are acknowledged but not implemented: a transactional outbox guaranteeing atomic database-and-event commit (Section 4.3), a dead-letter exchange for unrecoverable messages (Section 3.6), and Gateway-level rate limiting. The present implementation prioritizes architectural completeness across the five services over per-service operational depth.

6.3 Future Work

The limitations enumerated above suggest two direct future directions, each building on infrastructure already developed.

Evaluation against anonymized real-world data. The most immediate priority is to revisit the detection and risk classification ensembles with anonymized real-world transaction data once access becomes available, testing the external validity acknowledged in Section 6.2 and surfacing divergence between the TÜİK-calibrated synthesis assumptions and production transaction streams.

Online learning and model drift management. The current pipeline retrains offline from accumulated historical data. A future direction is to introduce incremental retraining and concept-drift detection alongside the existing inference path, allowing deployed models to adapt to gradual behavioral shifts. This subsumes a streaming-style ingestion pipeline at higher event throughput, since drift detection requires continuous evaluation against incoming events.

References

- Alabi, M., & Ang, A. W. (2024). AI-driven financial risk management: Detecting anomalies and predicting market trends. ResearchGate. <https://www.researchgate.net/publication/383276974>
- de Almeida, M. G., & Canedo, E. D. (2022). Authentication and authorization in microservices architecture: A systematic literature review. *Applied Sciences*, 12(6), 3023. <https://doi.org/10.3390/app12063023>
- Dhandapani, A. (2025). Microservices architecture in financial services: Enabling real-time transaction processing and enhanced scalability. *European Journal of Computer Science and Information Technology*, 13(32), 145–159. <https://doi.org/10.37745/ejcsit.2013/vol13n32145159>
- Fadul, A. M. A. (2023). Anomaly detection based on Isolation Forest and Local Outlier Factor [Master's thesis]. ResearchGate. <https://www.researchgate.net/publication/374332567>
- John, H., & Naaz, S. (2019). Credit card fraud detection using Local Outlier Factor and Isolation Forest. *International Journal of Computer Sciences and Engineering*, 7(4), 1060–1064. <https://www.researchgate.net/publication/335809102>
- Laigner, R., Almeida, A. C., Assunção, W. K. G., & Zhou, Y. (2025). An empirical study on challenges of event management in microservice architectures. *ACM Transactions on Software Engineering and Methodology*. <https://doi.org/10.1145/3776581>
- Li, Y., Fan, S., & Wang, H. (2025). Machine learning-based identification of anomalous trading behavior patterns among Asia-Pacific investors in U.S. securities markets. *Spectrum of Research*. <https://spectrumofresearch.com/index.php/sr/article/view/37>
- Marieska, M. D., Yunanta, A., Aulia, H., Utami, A. S., & Rizqie, M. Q. (2025). Performance comparison of monolithic and microservices architectures in handling high-volume transactions. *Jurnal RESTI*, 9(3). <https://doi.org/10.29207/resti.v9i3.6183>
- Odofin, O. T., Owoade, S., Ogbuefi, E., Ogeawuchi, J. C., Adanigbo, O. S., & Gbenle, T. P. (2022). Integrating event-driven architecture in fintech operations using Apache Kafka and RabbitMQ systems. *International Journal of Multidisciplinary Research and Growth Evaluation*, 3(4), 635–643. <https://doi.org/10.54660/IJMRGE.2022.3.4.635-643>
- Odunlami, B. G., & Nwonu, B. (2025). Credit risk prediction using ensemble and linear machine learning models. ResearchGate. <https://www.researchgate.net/publication/394480756>

Pincirolì, R., Aletti, A., & Trubiani, C. (2023). Performance modeling and analysis of design patterns for microservice systems. In *Proceedings of the 2023 IEEE 20th International Conference on Software Architecture (ICSA)* (pp. 35–46). IEEE. <https://doi.org/10.1109/ICSA56044.2023.00012>

Purwar, A., & Manju. (2023). Credit card fraud detection using XGBoost for imbalanced data set. In *Proceedings of the 2023 Fifteenth International Conference on Contemporary Computing (IC3 2023)* (pp. 216–219). Association for Computing Machinery. <https://doi.org/10.1145/3607947.3607986>

Sarna, N. J., Rithen, F. A., Jui, U. S., Belal, S., Amin, A., Oishee, T. K., & Islam, A. K. M. M. (2025). AI driven fraud detection models in financial networks: A comprehensive systematic review. *IEEE Access*. <https://doi.org/10.1109/ACCESS.2025.3596060>

Shanaa, M., & Abdallah, S. (2025). A hybrid anomaly detection framework combining supervised and unsupervised learning for credit card fraud detection. *F1000Research*, 14, 664. <https://www.researchgate.net/publication/393470403>

Sinaga, N., & Machfud, S. (2025). Comparative analysis of Random Forest and XGBoost algorithms for credit risk classification. *Inotera*, 10(2). <https://doi.org/10.31572/inotera.Vol10.Iss2.2025.ID556>

Türkiye İstatistik Kurumu. (2025). Hanehalkı tüketim harcamasının türlerine göre dağılımı, 2024 [İstatistiksel tablo]. <https://data.tuik.gov.tr/Kategori/GetKategori?p=Gelir-Yasam-Tuketim-ve-Yoksulluk-107>

Yakhin, S. (2025). Comparative review of Clean Architecture and Vertical Slice Architecture approaches for enterprise .NET applications. *International Journal of Advanced Artificial Intelligence Research*, 2(12). <https://doi.org/10.55640/ijaair-v02i12-01>